

FC7xxx I2C User Manual

Rev.0.4

Revision History

Revision	Date	Changes
0.1	2023/07/14	Initial release for MCAL V0.1.0
0.3	2023/10/20	Updated for MCAL V0.3.0
0.4	2023/11/22	Updated for MCAL V0.4.0

Table of Contents

Revision History.....	2
Table of Contents	3
Chapter 1 I2C Introduction	5
1.1 Requirement Tracing.....	5
1.2 Hardware Summary.....	5
Chapter 2 Software Design	6
2.1 Rejected Requirements	6
2.2 File Structure.....	6
2.3 Macros	6
2.3.1 Macros in CDD_I2c.h	6
2.3.2 Macros in I2c_version.h	8
2.3.3 Macros in CDD_I2c_Cfg.h.....	8
2.3.4 Macros in CDD_I2c_Types.h	9
2.4 Enums.....	11
2.4.1 Enums in CDD_I2c_Types.h.....	11
2.5 Typedefs.....	11
2.6 Structures.....	11
2.6.1 I2c_ConfigType	11
2.6.2 I2c_InitChannelConfigType.....	11
2.6.3 FCIIIC_InitType.....	12
2.6.4 FCIIIC_RegCfgType.....	12
2.6.5 I2c_RequestType	13
2.6.6 FCIIIC_Type	13
2.7 API Functions	15
2.7.1 Functions in CDD_I2c.h.....	15
2.8 Hardware Functions	18
2.8.1 Functions in CDD_I2c_Hw.h	18
2.9 API Sequence Diagram	21
2.9.1 Initialization of the I2C Driver.....	21
2.9.2 Modes transitions	21
2.9.3 SyncTransmit	22
2.9.4 AsyncTransmit	23
2.9.5 StartListening.....	24
Chapter 3 Tresos Configuration Items	25
3.1 Container Inclusion Relation.....	25

3.2	Containers and Variables	26
3.2.1	IMPLEMENTATION_CONFIG_VARIANT	26
3.2.2	GeneralConfiguration	26
3.2.3	I2cEcucPartitionRef	29
3.2.4	I2cChannel.....	30
3.2.5	I2cMasterConfiguration	31
3.2.6	I2cSlaveConfiguration	36
3.2.7	CommonPublishedInformation	39
Chapter 4 Configuration Guides.....		42
4.1	Configuration Item Constraint	42
4.2	I2C Usage Common Steps	44

Chapter 1 I2C Introduction

1.1 Requirement Tracing

I2c is a Complex Device Driver (CDD), so there are no AUTOSAR requirements regarding this module. It has vendor-specific requirements and implementation.

1.2 Hardware Summary

The I2C driver is implemented as an Autosar complex device driver. It uses the FCI2C hardware peripherals to provides support for implementing the I2C protocol. The I2C driver implements both master and slave mode for FCI2C channels.

The driver offers a hardware independent API to the upper layer that can be used to configure the I2C and initiate synchronous and asynchronous data transfers. Asynchronous transfer for a master channel may use either interrupts or DMA. The slave operates only in asynchronous mode.

Hardware and software settings can be configured using an Autosar standard configuration tool. The information required for an I2C data transfer will be configured in a data structure that will be sent as parameter to the API of the driver.

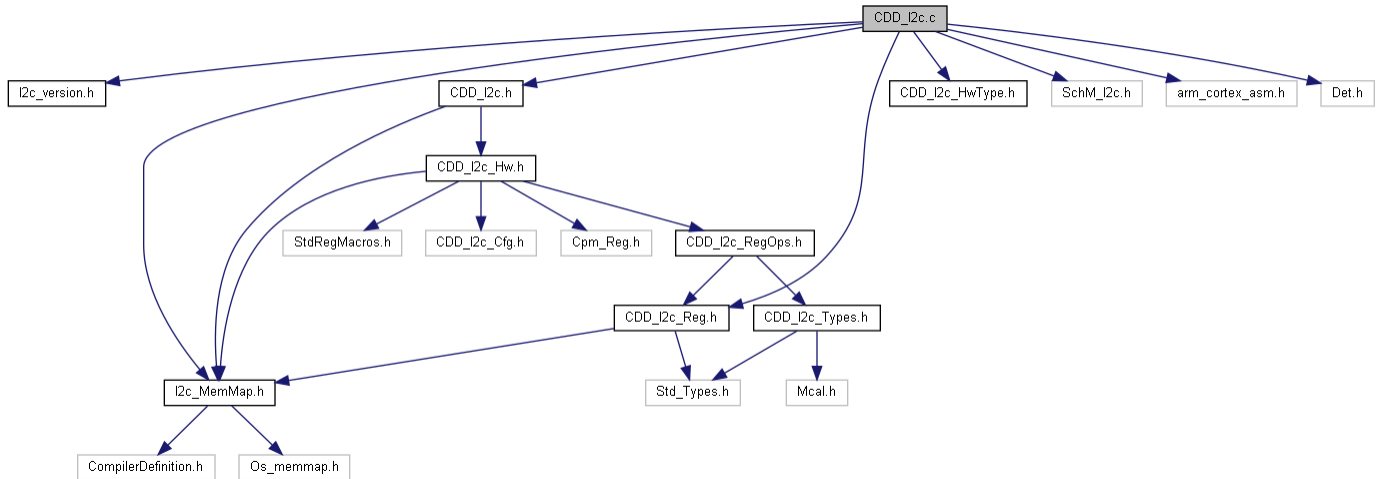
The driver reports errors to the error manager as defined in AUTOSAR.

Chapter 2 Software Design

2.1 Rejected Requirements

None

2.2 File Structure



2.3 Macros

2.3.1 Macros in CDD_I2c.h

- **#define I2C_E_PARAM_JOB** ((uint8)0x01U)
API service called with wrong parameter job.
- **#define I2C_E_PARAM_SEQUENCE** ((uint8)0x02U)
API service called with wrong parameter sequence.
- **#define I2C_E_PARAM_LENGTH** ((uint8)0x03U)
API service called with wrong parameter length.
- **#define I2C_E_PARAM_UNIT** ((uint8)0x04U)
API service called with wrong parameter channel.
- **#define I2C_E_PARAM_ADDRESS** ((uint8)0x05U)
API service called with wrong parameter address.
- **#define I2C_E_PARAM_POINTER** ((uint8)0x06U)
API service called with an unexpected value for the pointer.

- **#define I2C_E_UNINIT** ((uint8)0x07U)
API service used without module initialization.
- **#define I2C_E_BUSY** ((uint8)0x08U)
API service used while the I2C Driver is busy.
- **#define I2C_E_ALREADY_INITIALIZED** ((uint8)0x09U)
API I2C_Init service called while the I2C Driver has already been initialized.
- **#define I2C_E_INV_CTRL_IDX** ((uint8)0x0AU)
API service called with wrong core id.
- **#define I2C_E_TIMEOUT_FAILURE** ((uint8)0x01U)
Bus is busy for a period of time larger than the configured timeout.
- **#define I2C_E_NACK_RECEIVED** ((uint8)0x02U)
Error is reported if NACK was received.
- **#define I2C_E_ARBITRATION_FAILURE** ((uint8)0x03U)
Error is reported if the master loses arbitration. This usually happens if the SDA is stuck low or another master has won the arbitration procedure.
- **#define I2C_E_FIFO_HANDLING** ((uint8)0x04U)
Error is reported in case of FIFO underflow or overflow.
- **#define I2C_E_BUS_FAILURE** ((uint8)0x05U)
Error is reported if the SCL line is stuck low.
- **#define I2C_INIT_ID** ((uint8)0x00U)
API service ID for I2c_Init function.
- **#define I2C_DEINIT_ID** ((uint8)0x01U)
API service ID for I2c_DeInit function.
- **#define I2C_SYNCTRANSMIT_ID** ((uint8)0x02U)
API service ID for I2c_SyncTransmit function.
- **#define I2C_ASYNCTRANSMIT_ID** ((uint8)0x03U)

API service ID for I2c_AsyncTransmit function.

- **#define I2C_GETSTATUS_ID** ((uint8)0x04U)
API service ID for I2c_GetStatus function.
- **#define I2C_STARTLISTENING_ID** ((uint8)0x06U)
API service ID for I2c_StartListening function.
- **#define I2C_GETVERSIONINFO_ID** ((uint8)0x0AU)
API service ID for I2c_GetVersionInfo function.

2.3.2 Macros in I2c_version.h

- **#define I2C_AR_RELEASE_MAJOR_VERSION** 4
- **#define I2C_AR_RELEASE_MINOR_VERSION** 6
- **#define I2C_AR_RELEASE_REVISION_VERSION** 0
- **#define I2C_SW_MAJOR_VERSION** 0
- **#define I2C_SW_MINOR_VERSION** 4
- **#define I2C_SW_PATCH_VERSION** 0
- **#define I2C_VENDOR_ID** 174
- **#define I2C_MODULE_ID** 255

2.3.3 Macros in CDD_I2c_Cfg.h

- **#define I2C_CFG_VENDOR_ID_C** 174
- **#define I2C_CFG_AR_RELEASE_MAJOR_VERSION_C** 4
- **#define I2C_CFG_AR_RELEASE_MINOR_VERSION_C** 6
- **#define I2C_CFG_AR_RELEASE_REVISION_VERSION_C** 0

- `#define I2C_CFG_SW_MAJOR_VERSION` 0
- `#define I2C_CFG_SW_MINOR_VERSION` 4
- `#define I2C_CFG_SW_PATCH_VERSION` 0
- `#define I2C_PRECOMPILE_SUPPORT` (STD_OFF)
- `#define I2C_MAX_MODULES` 2
- `#define I2C_MAX_CORE_ID` 3
- `#define I2C_TIMEOUT_LOOPS` 1000U
- `#define I2C_DMA_USED` (STD_ON)
- `#define I2C_VERSION_INFO_API` (STD_ON)
- `#define I2C_DEV_ERROR_DETECT` (STD_ON)
- `#define I2C_MULTICORE_SUPPORT` (STD_ON)
- `#define FCI2C_0` 0U
- `#define FCI2C_1` 1U

2.3.4 [Macros in CDD_I2c_Types.h](#)

- `#define I2C_SLAVE_MODE` ((uint8)0x00U)
I2C slave mode.
- `#define I2C_MASTER_MODE` ((uint8)0x01U)
I2C master mode.
- `#define I2C_MODE_UNINIT` ((uint8)0x02U)
I2C uninit.
- `#define I2C_E_PIN_LOW_TIMEOUT` ((uint8)0x01U)
SCL or SDA pin low timeout error.

- **#define I2C_E_MASTER_FIFO_ERROR** ((uint8)0x02U)
Master transmit or receive fifo full.
- **#define I2C_E_MASTER_ARBITRATION_LOST** ((uint8)0x03U)
Master bus arbitration error.
- **#define I2C_E_MASTER_UNEXPECTED_NACK** ((uint8)0x04U)
Master NACK error.
- **#define I2C_E_MASTER_RX_OVERFLOW** ((uint8)0x05U)
Master receive fifo full.
- **#define I2C_E_MASTER_TX_UNDERFLOW** ((uint8)0x06U)
Master transmit fifo empty.
- **#define I2C_DIR_READ** (0x1U)
IIC read data.
- **#define I2C_DIR_WRITE** (0x0U)
IIC write data.
- **#define I2C_DIR_MASK** (0x1U)
IIC direction mask.

2.4 Enums

2.4.1 Enums in CDD_I2c_Types.h

2.2.6.1 Enums in CDD_I2c_Types.h

Enumeration	I2c_ChannelStatusType	
Description	IIC channel status enum.	
Values	Value	Description
	I2C_CH_IDLE = 0	I2C channel is idle
	I2C_CH_SENDING = 1	I2C channel is sending
	I2C_CH_RECEIVING = 2	I2C channel is receiving
	I2C_CH_LISTENING = 3	I2C channel is listening for master to send
	I2C_CH_ERROR_PRESENT = 4	I2C channel is error present
	I2C_CH_CLOSED = 5	I2C channel is closed

2.5 Typedefs

None

2.6 Structures

2.6.1 I2c_ConfigType

Structure	I2c_ConfigType
Description	This structure contains IIC initialization data.
Diagram	<pre> classDiagram class FCIIIC_RegCfgType class FCIIIC_InitType class I2c_InitChannelConfigType class I2c_ConfigType FCIIIC_RegCfgType --> I2c_InitChannelConfigType : +FCIIIC_reg FCIIIC_InitType --> I2c_InitChannelConfigType : +FCIIIC_Config I2c_InitChannelConfigType --> I2c_ConfigType : +I2c_pParmConfig </pre>
Data Fields	- uint8 I2c_MaxHwUnit The maximum number of configured I2C Hw unit in the current configuration structure. - const I2c_InitChannelConfigType(* I2c_pParmConfig)[] Pointer to I2c hardware unit configuration

2.6.2 I2c_InitChannelConfigType

Structure	I2c_InitChannelConfigType
Description	This structure contains the hardware channel configuration parameters
Diagram	<pre> classDiagram class FCIIIC_RegCfgType class FCIIIC_InitType class I2c_InitChannelConfigType FCIIIC_RegCfgType --> I2c_InitChannelConfigType : +FCIIIC_reg FCIIIC_InitType --> I2c_InitChannelConfigType : +FCIIIC_Config </pre>
Data Fields	uint8 I2c_CoreId

	<i>Reference ECU partition.</i> • const FCIIIC_InitType * tFCI2c_Config Incidentates init mode. • const FCIIIC_RegCfgType * tFCI2c_reg Init registers for I2c.
--	--

2.6.3 FCIIIC_InitType

Structure	FCIIIC_InitType
Description	IIC initial data define.
Diagram	N/A
Data Fields	• uint8 u8MasterMode bMasterMode=1 master mode • uint8 u8SlaveAddr If bMasterMode=0, this is used, and address format is 7bits , not ended with R/W bit • uint8 u8HwIndex Define the index select by user

2.6.4 FCIIIC_RegCfgType

Structure	FCIIIC_RegCfgType
Description	IIC initial register struct define.
Diagram	N/A
Data Fields	• uint32 MCR Master Control. • uint32 MDER MDER Receive and transmit data dma enable. • uint32 MCFGR0 Master Configuration 0. • uint32 MCFGR1 Master Configuration 1. • uint32 MCFGR2 Master Configuration 2. • uint32 MCFGR3 Master Configuration 3. • uint32 MCCR Master Clock Configuration. • uint32 MFCR MFCR Receive & transmit FIFO Watermark. • uint32 SCR Slave Control. • uint32 SCFGR1 Slave Configuration 1. • uint32 SCFGR2

	Slave Configuration 2. - uint32 SAMR Slave Address Match - uint8 u8TxDMAChannel. DMA channel used for transmit data. - uint8 u8RxDMAChannel DMA channel used for receive data.
--	--

2.6.5 I2c_RequestType

Structure	I2c_RequestType
Description	IIC request struct define.
Diagram	N/A
Data Fields	- uint8 u8SlaveAddress 7bit Slave address. - uint8 u8Direction Read or write direction. - uint8 u8Len the length of read or write bytes. - uint8 * BufferPtr Pointer to buffer.

2.6.6 FCIIIC_Type

Structure	FCIIIC_Type
Description	This structure contains the registers layout of FCIIIC.
Diagram	N/A
Data Fields	- uint8 RESERVED_0 [16] Reserved bits. __IO uint32 MCR Master Control, offset: 0x10. __IO uint32 MSR Master Status, offset: 0x14. __IO uint32 MIER Master Interrupt Enable, offset: 0x18. __IO uint32 MDER Master DMA Enable, offset: 0x1C. __IO uint32 MCFGR0 Master Configuration 0, offset: 0x20. __IO uint32 MCFGR1 Master Configuration 1, offset: 0x24. __IO uint32 MCFGR2 Master Configuration 2, offset: 0x28. __IO uint32 MCFGR3

	Master Configuration 3, offset: 0x2C. • uint8 RESERVED_1 [16] Reserved bits. • __IO uint32 MDMR Master Data Match, offset: 0x40. • uint8 RESERVED_2 [4] Reserved bits. • __IO uint32 MCCR Master Clock Configuration, offset: 0x48. • uint8 RESERVED_3 [12] Reserved bits. • __IO uint32 MFCR Master FIFO Control, offset: 0x58. • __I uint32 MFSR Master FIFO Status, offset: 0x5C. • __O uint32 MTDR Master Transmit Data, offset: 0x60. • uint8 RESERVED_4 [12] Reserved bits. • __I uint32 MRDR Master Receive Data, offset: 0x70. • uint8 RESERVED_5 [16] Reserved bits. • uint8 RESERVED_6 [140] Reserved bits. • __IO uint32 SCR Slave Control, offset: 0x110. • __IO uint32 SSR Slave Status, offset: 0x114. • __IO uint32 SIER Slave Interrupt Enable, offset: 0x118. • __IO uint32 SDER Slave DMA Enable, offset: 0x11C. • uint8 RESERVED_7 [4] Reserved bits. • __IO uint32 SCFGR1 Slave Configuration 1, offset: 0x124. • __IO uint32 SCFGR2 Slave Configuration 2, offset: 0x128. • uint8 RESERVED_8 [20] Reserved bits. • __IO uint32 SAMR Slave Address Match, offset: 0x140. • uint8 RESERVED_9 [12]
--	--

	Reserved bits. • __I uint32 SASR Slave Address Status, offset: 0x150. • __IO uint32 STAR Slave Transmit ACK, offset: 0x154. • uint8 RESERVED_10 [8] reserved bits • __O uint32 STDR Slave Transmit Data, offset: 0x160 • uint8 RESERVED_11 [12] Reserved bits. • __I uint32 SRDR Slave Receive Data, offset: 0x170.
--	---

2.7 API Functions

2.7.1 Functions in CDD_I2c.h

2.2.6.1 void I2c_Init(const I2c_ConfigType * pl2cCfg)

Function	void I2c_Init (const I2c_ConfigType * pl2cCfg)	
Description	This function is used to initial IIC.	
Diagram	N/A	
Parameters	Parameter	Description
	[in] pl2cCfg	The pointer of initialize parameters. Variant PostBuid mode the parameter is generated by CDD_I2c_PBcfg.c file
Returns	N/A	

2.2.6.2 void I2c_Delnit (void)

Function	void I2c_Delnit (void)	
Description	This function is used to de-initial IIC.	
Diagram	N/A	
Parameters	N/A	
Returns	N/A	

2.2.6.3 I2c_ChannelStatusType I2c_GetStatus(uint8 u8Channel1)

Function	I2c_ChannelStatusType I2c_GetStatus(uint8 u8Channel1)	
Description	This function is used to get channel status.	
Diagram	N/A	
Parameters	Parameter	Description
	[in] u8Channel	The IIC channel.
Returns	Channel status enum value.	

2.2.6.4 void I2c_GetVersionInfo (Std_VersionInfoType *pVersionInfo)

Function	void I2c_GetVersionInfo (Std_VersionInfoType *pVersionInfo)	
Description	This function is used to get IIC version information.	
Diagram	N/A	
Parameters	Parameter	Description
	[out] pVersionInfo	Pointer to where to store the version information of this module.
Returns	N/A	

2.2.6.5 Std_ReturnType I2c_SyncTransmit(uint8 u8Channel, const I2c_RequestType *pRequestPtr)

Function	Std_ReturnType I2c_SyncTransmit(uint8 u8Channel, const I2c_RequestType *pRequestPtr)	
Description	This function is used for master to send or receive data by Synchronous communication mode.	
Diagram	N/A	
Parameters	Parameter	Description
	[in] u8Channel	The IIC channel.
	[in] pRequestPtr	It's a pointer contains IIC instance, direction, the length of data and buffer address
Returns	E_OK is ok, others are not ok	

2.2.6.6 Std_ReturnType I2c_AsyncTransmit(uint8 u8Channel, const I2c_RequestType *pRequestPtr)

Function	Std_ReturnType I2c_AsyncTransmit(uint8 u8Channel, const I2c_RequestType *pRequestPtr)	
Description	This function is used for master to send or receive data by asynchronous communication mode.	
Diagram	N/A	
Parameters	Parameter	Description
	[in] u8Channel	The IIC channel.
	[in] pRequestPtr	It's a pointer contains IIC instance, direction, the length of data and buffer address
Returns	E_OK is ok, others are not ok	

2.2.6.7 Std_ReturnType I2c_StartListening(uint8 u8Channel, const I2c_RequestType *pRequestPtr)

Function	Std_ReturnType I2c_StartListening(uint8 u8Channel, const I2c_RequestType *pRequestPtr)	
Description	This function is used for slave to send or receive data.	
Diagram	N/A	
Parameters	Parameter	Description
	[in] u8Channel	The IIC channel.
	[in] pRequestPtr	It's a pointer contains IIC instance, direction, the length of data and buffer address
Returns	E_OK is ok, others are not ok	

2.2.6.8 void FCI2C_0_ISR(void)

Function	void FCI2C_0_ISR(void)
Description	This function is used to process interrupt for FCIIC0.

Diagram	N/A
Parameters	N/A
Returns	N/A

2.2.6.9 void FCI2C_1_ISR(void)

Function	void FCI2C_1_ISR(void)
Description	This function is used to process interrupt for FCIIC1.
Diagram	N/A
Parameters	N/A
Returns	N/A

2.2.6.10 void FCIIC0_DMATxIsr(void);

Function	void FCIIC0_DMATxIsr(void);
Description	This function is used to process interrupt for FCIIC0 DMA Tx.
Diagram	N/A
Parameters	N/A
Returns	N/A

2.2.6.11 void FCIIC0_DMARxIsr(void);

Function	void FCIIC0_DMARxIsr(void);
Description	This function is used to process interrupt for FCIIC0 DMA Rx.
Diagram	N/A
Parameters	N/A
Returns	N/A

2.2.6.12 void FCIIC1_DMATxIsr(void);

Function	void FCIIC1_DMATxIsr(void);
Description	This function is used to process interrupt for FCIIC1 DMA Tx.
Diagram	N/A
Parameters	N/A
Returns	N/A

2.2.6.13 void FCIIC1_DMARxIsr(void);

Function	void FCIIC1_DMARxIsr(void);
Description	This function is used to process interrupt for FCIIC1 DMA Rx.
Diagram	N/A
Parameters	N/A
Returns	N/A

2.8 Hardware Functions

2.8.1 Functions in CDD_I2c_Hw.h

2.2.6.1 Std_ReturnType FCIIC_Init_Master(uint8 u8I2cIdx, uint8 u8CoreId, const FCIIC_RegCfgType *pInitReg)

Function	Std_ReturnType FCIIC_Init_Master(uint8 u8I2cIdx, uint8 u8CoreId, const FCIIC_RegCfgType *pInitReg)	
Description	This function is used to initialize IIC master registers.	
Diagram	N/A	
Parameters	Parameter	Description
	[in] u8I2cIdx	IIC index, 0 or 1
	[in] u8CoreId	Core Id corresponding to IIC hardware unit.
	[in] pInitCfg	The pointer of IIC parameters need to config
Returns	E_OK is ok, others are not ok	
Referenced By	I2c_Init()	

2.2.6.2 Std_ReturnType FCIIC_DeInit_Master(uint8 u8I2cIdx)

Function	Std_ReturnType FCIIC_DeInit_Master(uint8 u8I2cIdx)	
Description	This function is used to de-initialize IIC master registers.	
Diagram	N/A	
Parameters	Parameter	Description
	[in] u8I2cIdx	IIC index, 0 or 1
Returns	E_OK is ok, others are not ok	
Referenced By	I2c_DeInit()	

2.2.6.3 Std_ReturnType FCIIC_Init_Slave(uint8 u8I2cIdx, uint8 u8CoreId, const FCIIC_RegCfgType *pInitReg)

Function	Std_ReturnType FCIIC_Init_Slave(uint8 u8I2cIdx, uint8 u8CoreId, const FCIIC_RegCfgType *pInitReg)	
Description	This function is used to initialize IIC slave registers.	
Diagram	N/A	
Parameters	Parameter	Description
	[in] u8I2cIdx	IIC index, 0 or 1
	[in] u8CoreId	Core Id corresponding to IIC hardware unit.
	[in] pInitCfg	The pointer of IIC parameters need to config
Returns	E_OK is ok, others are not ok	
Referenced By	I2c_Init()	

2.2.6.4 Std_ReturnType FCIIC_DeInit_Slave(uint8 u8I2cIdx)

Function	Std_ReturnType FCIIC_DeInit_Slave(uint8 u8I2cIdx)	
Description	This function is used to de-initialize IIC slave registers.	
Diagram	N/A	
Parameters	Parameter	Description
	[in] u8I2cIdx	IIC index, 0 or 1
Returns	E_OK is ok, others are not ok	
Referenced By	I2c_DeInit()	

2.2.6.5 Std_ReturnType FCIIC_MasterSyncSend(uint8 u8Channel, uint8 u8I2cIdx, const I2c_RequestType *pRequest)

Function	Std_ReturnType FCIIC_MasterSyncSend(uint8 u8Channel, uint8 u8I2cIdx, const I2c_RequestType *pRequest)	
Description	IIC master synchronous send.	
Diagram	N/A	
Parameters	Parameter	Description
	[in] u8Channel	The IIC channel.
	[in] u8I2cIdx	IIC index, 0 or 1
	[in] pRequest	The pointer of request type
Returns	E_OK is ok, others are not ok	
Referenced By	I2c_SyncTransmit()	

2.2.6.6 Std_ReturnType FCIIC_MasterSyncReceive(uint8 u8Channel, uint8 u8I2cIdx, const I2c_RequestType *pRequest)

Function	Std_ReturnType FCIIC_MasterSyncReceive(uint8 u8Channel, uint8 u8I2cIdx, const I2c_RequestType *pRequest)	
Description	IIC master synchronous receive.	
Diagram	N/A	
Parameters	Parameter	Description
	[in] u8Channel	The IIC channel.
	[in] u8I2cIdx	IIC index, 0 or 1
	[inout] pRequest	The pointer of request type
Returns	E_OK is ok, others are not ok	
Referenced By	I2c_SyncTransmit()	

2.2.6.7 Std_ReturnType FCIIC_MasterAsyncSend(uint8 u8Channel, uint8 u8I2cIdx, const I2c_RequestType *pRequest)

Function	Std_ReturnType FCIIC_MasterAsyncSend(uint8 u8Channel, uint8 u8I2cIdx, const I2c_RequestType *pRequest)	
Description	IIC master Asynchronous send.	
Diagram	N/A	
Parameters	Parameter	Description
	[in] u8Channel	The IIC channel.
	[in] u8I2cIdx	IIC index, 0 or 1
	[in] pRequest	The pointer of request type
Returns	E_OK is ok, others are not ok	
Referenced By	I2c_AsyncTransmit()	

2.2.6.8 Std_ReturnType FCIIC_MasterAsyncReceive(uint8 u8Channel, uint8 u8I2cIdx, const I2c_RequestType *pRequest)

Function	Std_ReturnType FCIIC_MasterAsyncReceive(uint8 u8Channel, uint8 u8I2cIdx, const I2c_RequestType *pRequest)	
Description	IIC master Asynchronous receive.	

Diagram	N/A	
Parameters	Parameter	Description
	[in] u8Channel	The IIC channel.
	[in] u8I2cIdx	IIC index, 0 or 1
	[inout] pRequest	The pointer of request type
Returns	E_OK is ok, others are not ok	
Referenced By	I2c_AsyncTransmit()	

2.2.6.9 Std_ReturnType FCIIC_StartListening(uint8 u8Channel, uint8 u8I2cIdx, const I2c_RequestType *pRequest)

Function	Std_ReturnType FCIIC_StartListening(uint8 u8Channel, uint8 u8I2cIdx, const I2c_RequestType *pRequest)	
Description	IIC slave mode transmit or receive data.	
Diagram	N/A	
Parameters	Parameter	Description
	[in] u8Channel	The IIC channel.
	[in] u8I2cIdx	IIC index, 0 or 1
	[inout] pRequest	The pointer of request type
Returns	E_OK is ok, others are not ok	
Referenced By	I2c_StartListening()	

2.2.6.10 I2c_ChannelStatusType FCIIC_GetChannelStatus(uint8 u8I2cIdx)

Function	I2c_ChannelStatusType FCIIC_GetChannelStatus(uint8 u8I2cIdx)	
Description	IIC get current channel status.	
Diagram	N/A	
Parameters	Parameter	Description
	[in] u8I2cIdx	IIC index, 0 or 1
Returns	Channel status enum value	
Referenced By	I2c_GetStatus()	

2.2.6.11 void FCIIC_LL_IRQnHandler(uint8 u8I2cIdx)

Function	void FCIIC_LL_IRQnHandler(uint8 u8I2cIdx)	
Description	IIC interrupt handler.	
Diagram	N/A	
Parameters	Parameter	Description
	[in] u8I2cIdx	IIC index, 0 or 1
Returns	N/A	
Referenced By	I2c_FCIIC0_Isr()/I2c_FCIIC1_Isr()	

2.2.6.12 void FCIIC_LL_DMA_IRQnHandler(uint8 u8I2cIdx)

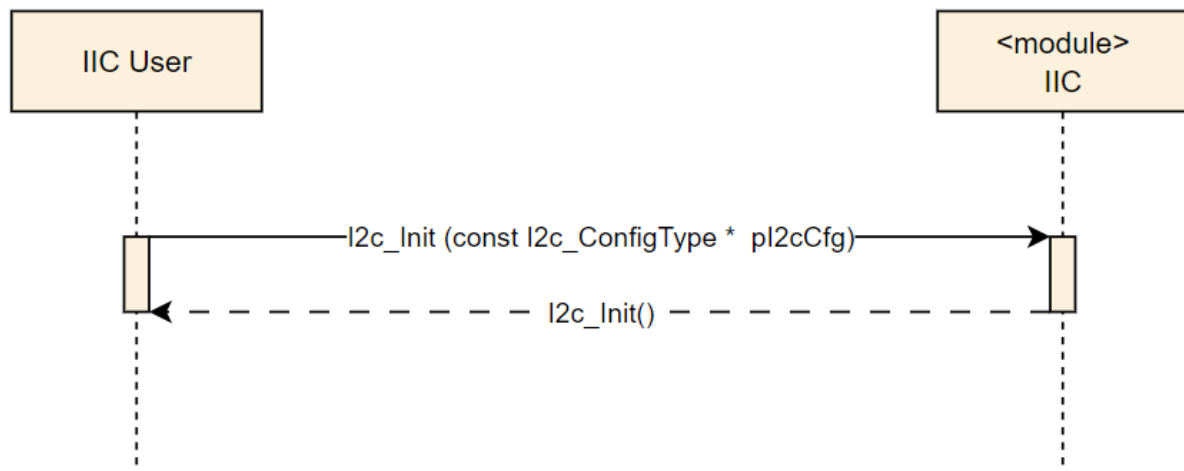
Function	void FCIIC_LL_DMA_IRQnHandler (uint8 u8I2cIdx)	
Description	IIC DMA interrupt handler.	

Diagram	N/A	
Parameters	Parameter	Description
	[in] u8I2cIdx	IIC index, 0 or 1
Returns	N/A	
Referenced By	DMA_Callback()	

2.9 API Sequence Diagram

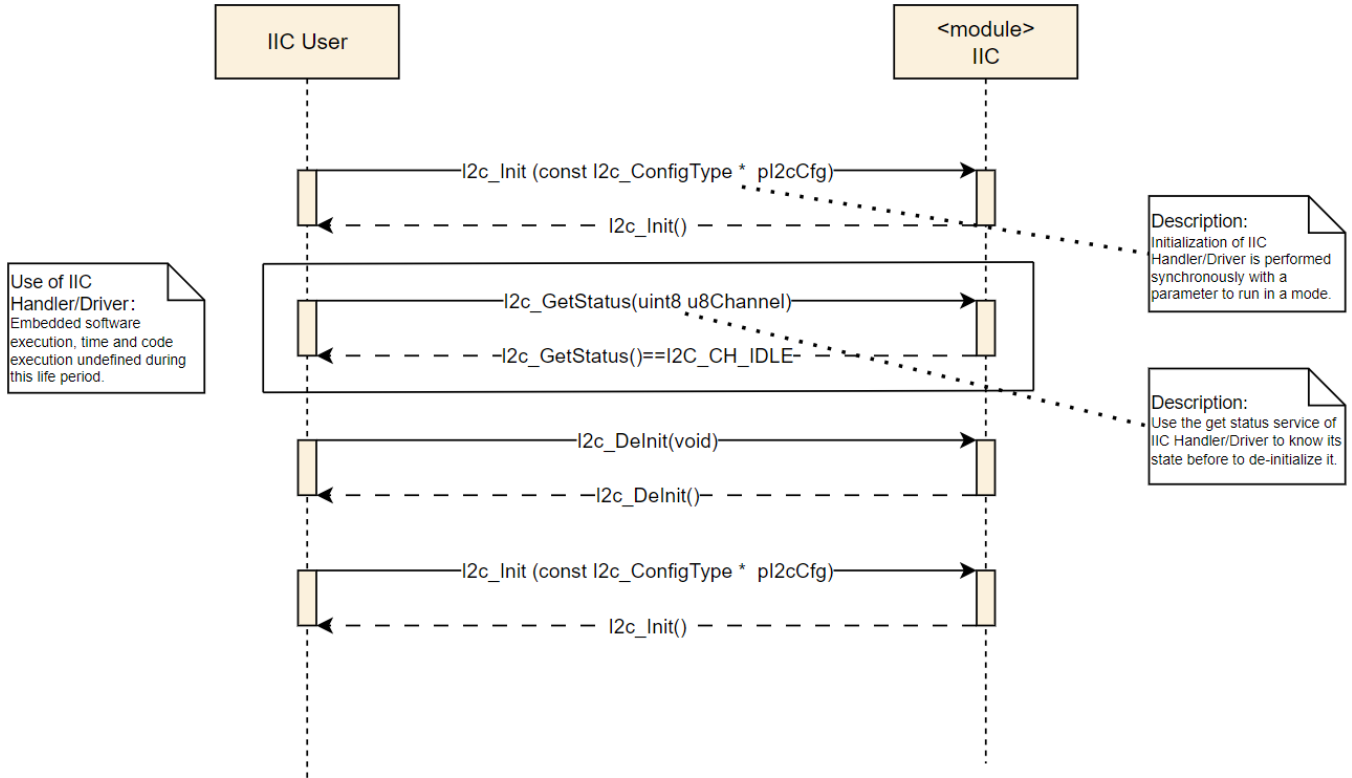
2.9.1 Initialization of the I2C Driver

The ECU State Manager (EcuM) is responsible for calling the initialization function.

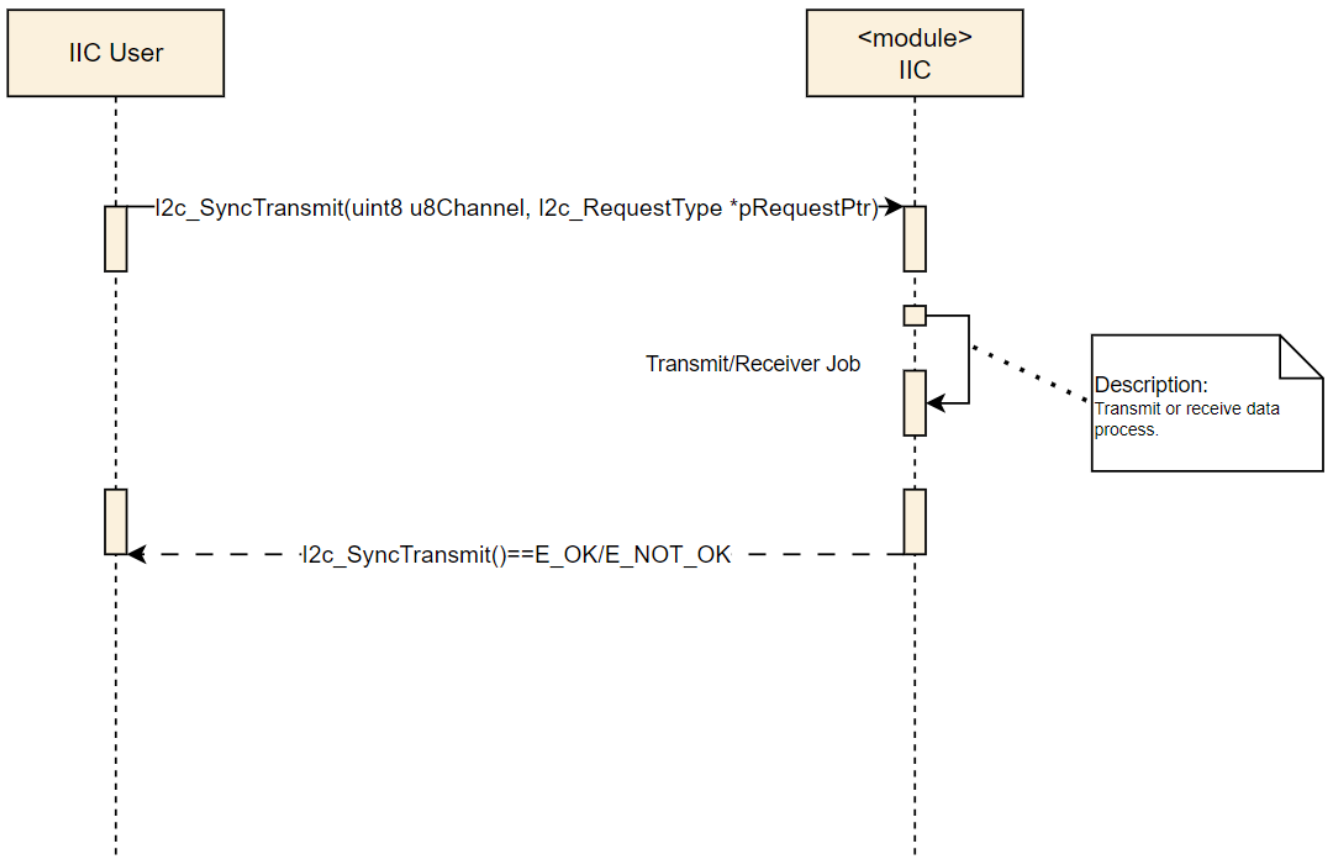


2.9.2 Modes transitions

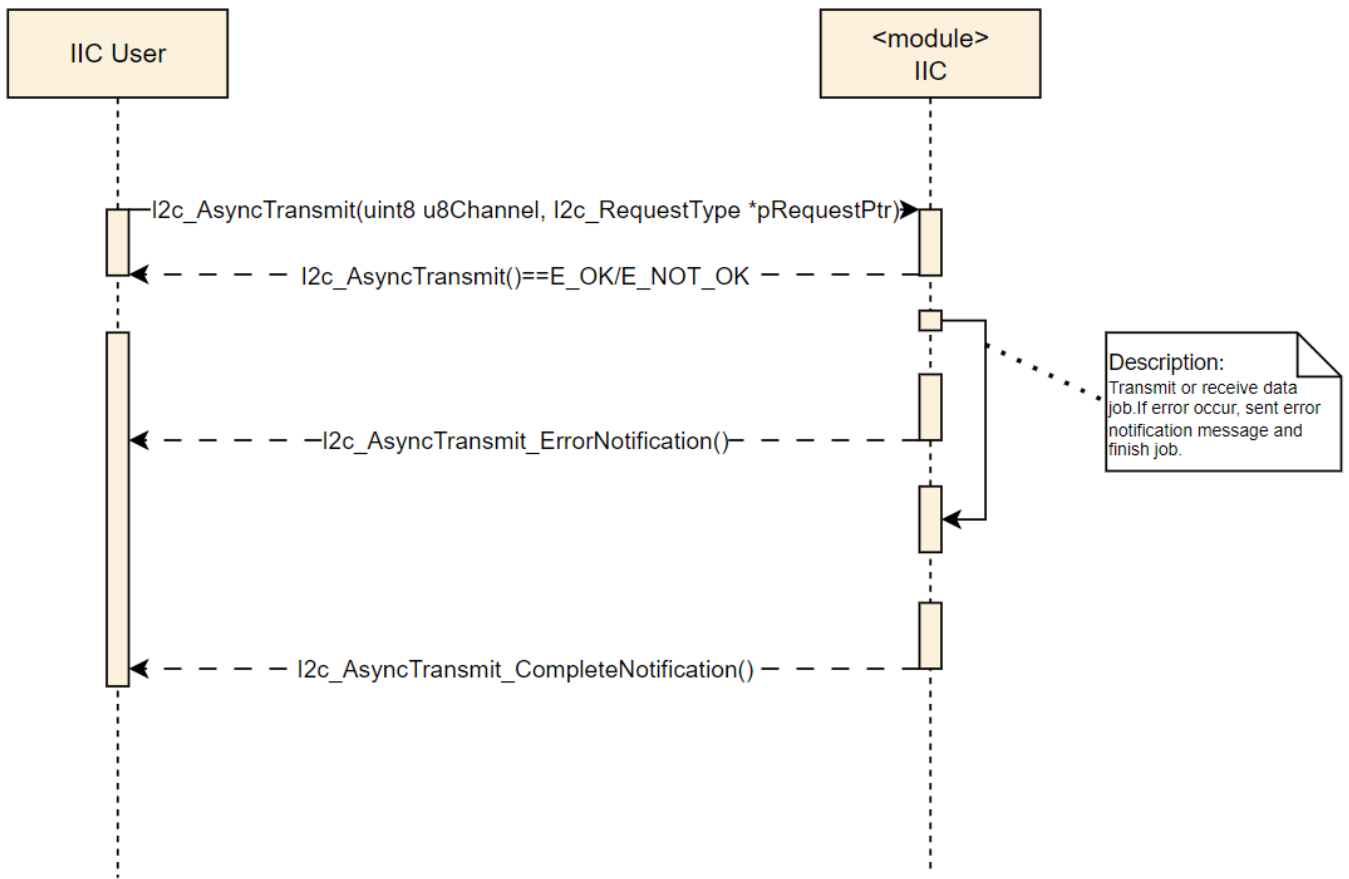
The following sequence diagram shows an example of an Init / DeInit calls for a running mode transition.



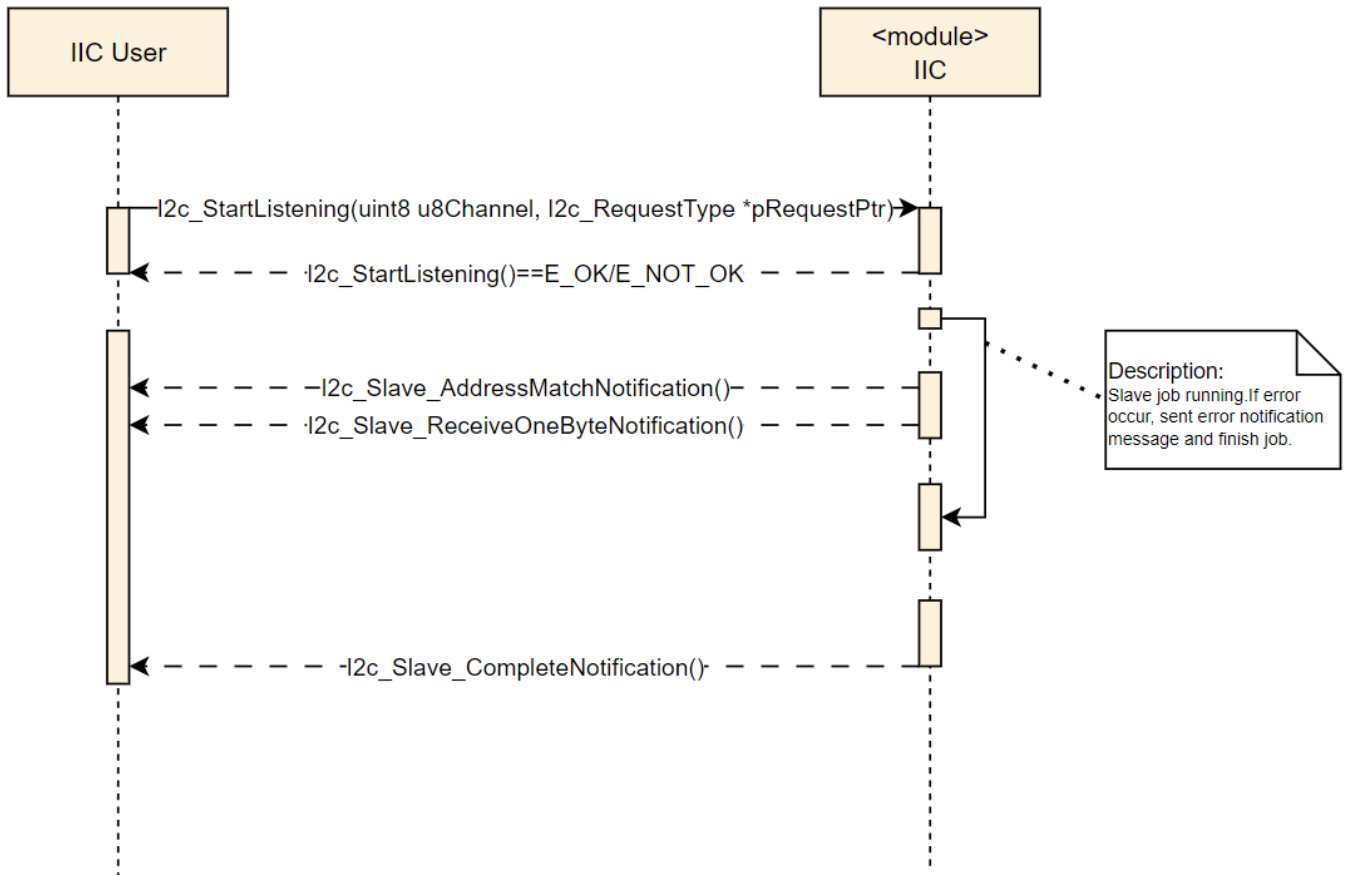
2.9.3 SyncTransmit



2.9.4 AsyncTransmit



2.9.5 StartListening



Chapter 3 Tresos Configuration Items

3.1 Container Inclusion Relation

The contain inclusion relation is shown as below:

GeneralConfiguration	CommonPublishedInformation
I2cDevErrorDetect	ArReleaseMajorVersion
I2cMulticoreSupport	ArReleaseMinorVersion
I2cTimeoutDuration	ArReleaseRevisionVersion
I2cVersionInfoApi	ModuleID
I2cDmaUsed	SwMajorVersion
I2cErrorNotification	SwMinorVersion
I2cMasterTransmitCompleteNotification	SwPatchVersion
I2cMasterReceiveCompleteNotification	VendorApiInfix
I2cSlaveAddressMatchNotification	Verdorld
I2cSlaveTransmitCompleteNotification	
I2cSlaveReceiveCompleteNotification	
I2cSlaveByteReceiveNotification	

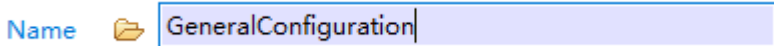


3.2 Containers and Variables

3.2.1 IMPLEMENTATION_CONFIG_VARIANT

Container	IMPLEMENTATION_CONFIG_VARIANT	
Description	N/A	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	Label	Config Variant
	Default	VariantPostBuild
	Range	VariantPostBuild VariantPreCompile
Returns	N/A	

3.2.2 GeneralConfiguration

Container	GeneralConfiguration
Description	This container contains the global configuration parameters of the Non-Autosar I2c driver.
Screenshot	

Properties	Property	Value
	Type	Container

3.2.2.1 I2cDevErrorDetect

Container	I2cDevErrorDetect	
Description	Switches the Development Error Detection and Notification ON or OFF.	
Screenshot	I2c Development Error Detection 	
Properties	Property	Value
	Type	BOOLEAN
	Origin	Flagchip
	Default	FALSE

3.2.2.2 I2cMulticoreSupport

Container	I2cMulticoreSupport	
Description	Switches multicore support ON or OFF.	
Screenshot	I2c Multicore Support 	
Properties	Property	Value
	Type	BOOLEAN
	Origin	Flagchip
	Default	FALSE

3.2.2.3 I2cTimeoutDuration

Container	I2cTimeoutDuration	
Description	Specifies the maximum number of loops for blocking function until a timeout is raised in short term wait loops.	
Screenshot	I2c Timeout Duration (1 -> 65535) 	
Properties	Property	Value
	Type	INTEGER
	Origin	Flagchip

3.2.2.4 I2cVersionInfoApi

Container	I2cVersionInfoApi	
Description	Switches the I2c_GetVersionInfo function ON or OFF.	
Screenshot	Provide I2c VersionInfo Api 	
Properties	Property	Value
	Type	BOOLEAN
	Origin	Flagchip
	Default	FALSE

3.2.2.5 I2cDmaUsed

Container	I2cDmaUsed	
-----------	------------	--

Description	Switches the I2c DMA function ON or OFF.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	Origin	Flagchip
	Default	FALSE

3.2.2.6 I2cErrorNotification

Container	I2cErrorNotification	
Description	Function name of callout. This function will be called if an error is detected.	
Screenshot		
Properties	Property	Value
	Type	FUNCTION-NAME
	Origin	Flagchip
	Default	NULL_PTR

3.2.2.7 I2cMasterTransmitCompleteNotification

Container	I2cMasterTransmitCompleteNotification	
Description	Function name of callout. This function will be called when a data transmission completes. (Only applicable in master mode!)	
Screenshot		
Properties	Property	Value
	Type	FUNCTION-NAME
	Origin	Flagchip
	Default	NULL_PTR

3.2.2.8 I2cMasterReceiveCompleteNotification

Container	I2cMasterReceiveCompleteNotification	
Description	Function name of callout. This function will be called when a data reception completes. (Only applicable in master mode!)	
Screenshot		
Properties	Property	Value
	Type	FUNCTION-NAME
	Origin	Flagchip
	Default	NULL_PTR

3.2.2.9 I2cSlaveAddressMatchNotification

Container	I2cSlaveAddressMatchNotification	
Description	Function name of callout. This function will be called if an address matching is detected.	
Screenshot		
Properties	Property	Value

	Type	FUNCTION-NAME
	Origin	Flagchip
	Default	NULL_PTR

3.2.2.10 I2cSlaveTransmitCompleteNotification

Container	I2cSlaveTransmitCompleteNotification	
Description	Function name of callout. This function will be called when a data transmission completes. (Only applicable in slave mode!)	
Screenshot		
Properties	Property	Value
	Type	FUNCTION-NAME
	Origin	Flagchip
	Default	NULL_PTR

3.2.2.11 I2cSlaveReceiveCompleteNotification

Container	I2cSlaveReceiveCompleteNotification	
Description	Function name of callout. This function will be called when a data reception completes. (Only applicable in slave mode!)	
Screenshot		
Properties	Property	Value
	Type	FUNCTION-NAME
	Origin	Flagchip
	Default	NULL_PTR

3.2.2.12 I2cSlaveByteReceiveNotification

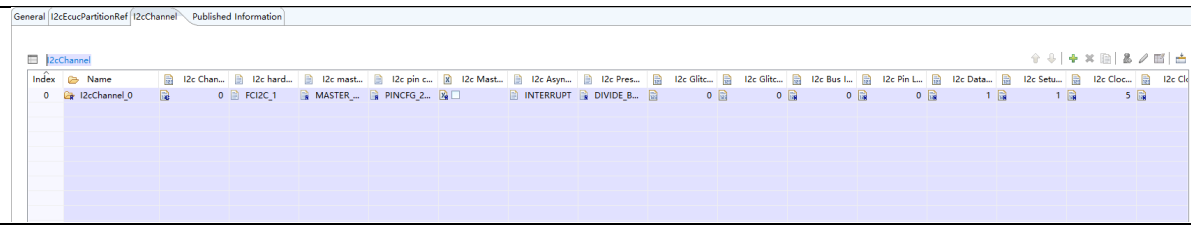
Container	I2cSlaveByteReceiveNotification	
Description	Function name of callout. This function will be called every time (after) a byte is received. (Only applicable in slave mode!)	
Screenshot		
Properties	Property	Value
	Type	FUNCTION-NAME
	Origin	Flagchip
	Default	NULL_PTR

3.2.3 I2cEcucPartitionRef

List	I2cEcucPartitionRef
Description	Maps the I2c driver to zero or multiple ECUC partitions to make the module's API available in this partition. The I2c driver will operate as an independent instance in each of the partitions.

Screenshot	GeneralI2cEcucPartitionRefI2cChannelPublished Information I2cEcucPartitionRef <table><thead><tr><th>Index</th><th>I2cEcucPartitionRef</th><th></th></tr></thead><tbody><tr><td>0</td><td>/EcuC/EcuC/EcucPartitionCollection_0/EcucPartition_1</td><td></td></tr><tr><td>1</td><td>/EcuC/EcuC/EcucPartitionCollection_0/EcucPartition_0</td><td></td></tr></tbody></table>		Index	I2cEcucPartitionRef		0	/EcuC/EcuC/EcucPartitionCollection_0/EcucPartition_1		1	/EcuC/EcuC/EcucPartitionCollection_0/EcucPartition_0	
	Index	I2cEcucPartitionRef									
0	/EcuC/EcuC/EcucPartitionCollection_0/EcucPartition_1										
1	/EcuC/EcuC/EcucPartitionCollection_0/EcucPartition_0										
Properties	Property	Value									
	Type	LIST									
	Origin	Flagchip									

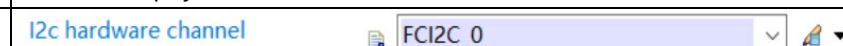
3.2.4 I2cChannel

List	I2cChannel	
Description	This container contains the configuration (parameters) of the I2c Controller(s).	
Screenshot		
Properties	Property	Value
	Type	SYMBOLIC-NAME-REFERENCE
	Origin	Flagchip

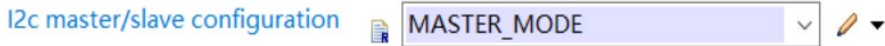
3.2.6.1 I2cChannelId

Variable	I2cChannelId	
Description	Identifies the I2c channel.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	Origin	Flagchip

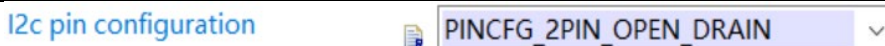
3.2.6.2 I2cHwChannel

Variable	I2cHwChannel	
Description	Selects the physical I2c Channel.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	Origin	Flagchip
	Default	FCI2C_0
	Range	FCI2C_0 FCI2C_1

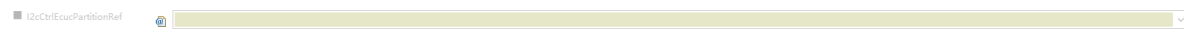
3.2.6.3 I2cMasterSlaveConfiguration

Variable	I2cMasterSlaveConfiguration	
Description	Selects the master slave configuration. Select whether the selected channel will be used as Master, Slave.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	Origin	Flagchip
	Default	MASTER_MODE
	Range	MASTER_MODE SLAVE_MODE

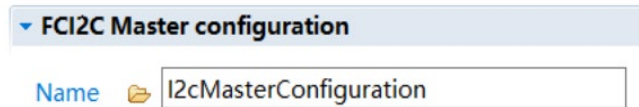
3.2.6.4 I2cPinConfiguration

Variable	I2cPinConfiguration	
Description	Selects the pin configuration. Configures the pin mode.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	Origin	Flagchip
	Default	PINCFG_2PIN_OPEN_DRAIN
	Range	PINCFG_2PIN_OPEN_DRAIN PINCFG_2PIN_PUSH_PULL

3.2.6.5 I2cCtrlEcucPartitionRef

Variable	I2cCtrlEcucPartitionRef	
Description	Maps the I2c controller to zero or one ECUC partition.	
Screenshot		
Properties	Property	Value
	Type	REFERENCE
	Origin	Flagchip

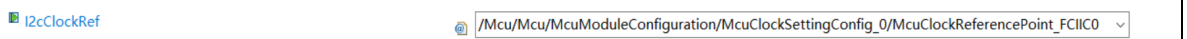
3.2.5 I2cMasterConfiguration

Container	I2cMasterConfiguration	
Description	This container contains the configuration (parameters) of the Master Configuration.	
Screenshot		
Properties	Property	Value
	Type	Container

3.2.5.1 I2cMasterEnabledInDebug

Container	I2cMasterEnabledInDebug	
Description	I2cMasterEnabledInDebug configures FCI2C_MCR[DBGEN] Check to enable the master in debug mode. Uncheck to disable it.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	Origin	Flagchip
	Default	FALSE

3.2.5.2 I2cClockRef

Container	I2cClockRef	
Description	Reference to the I2c clock source configuration, which is set in the MCU driver configuration.	
Screenshot		
Properties	Property	Value
	Type	REFERENCE
	Origin	Flagchip
	REF	ASPathDataOfSchema:/AUTOSAR/EcucDefs/Mcu/McuModuleConfiguration/McuClockSettingConfig/McuClockReferencePoint

3.2.5.3 I2cAsyncMode

Variable	I2cAsyncMode	
Description	Selects I2c Async mode.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	Origin	Flagchip
	Default	INTERRUPT
	Range	INTERRUPT DMA

3.2.5.4 I2cTxDmaChannel

Variable	I2cTxDmaChannel	
Description	Select logical DMA channel for I2c transive data.	
Screenshot		
Properties	Property	Value
	Type	REFERENCE
	Origin	Flagchip

	REF	ASPathDataOfSchema:/AUTOSAR/TOP-LEVEL- PACKAGES/I2c/ELEMENTS/I2c/I2cGlobalConfig/I2cChannel/I2cChannel _0/I2cMasterConfiguration/I2cTxDmaChannel
--	-----	--

3.2.5.5 I2cRxDmaChannel

Variable	I2cRxDmaChannel	
Description	Select logical DMA channel for I2c receive data.	
Screenshot		
Properties	Property	Value
	Type	REFERENCE
	Origin	Flagchip
	REF	ASPathDataOfSchema:/AUTOSAR/TOP-LEVEL- PACKAGES/I2c/ELEMENTS/I2c/I2cGlobalConfig/I2cChannel/I2cChannel _0/I2cMasterConfiguration/I2cRxDmaChannel

3.2.5.6 I2cPrescaler

Container	I2cPrescaler	
Description	PRESCALE: Configures FCI2C_MCFGR1[PRESCALE] Configures the clock prescaler used for all FCI2C master logic, except the digital glitch filters.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	Origin	Flagchip
	Default	DIVIDE_BY_1
	Range	DIVIDE_BY_1 DIVIDE_BY_2 DIVIDE_BY_4 DIVIDE_BY_8 DIVIDE_BY_16 DIVIDE_BY_32 DIVIDE_BY_64 DIVIDE_BY_128

3.2.5.7 I2cGlitchFilterSDA

Container	I2cGlitchFilterSDA	
Description	Glitch Filter SDA: Configures FCI2C_MCFGR2[FILTSDA] Configures the I2c master digital glitch filters for SDA input, a configuration of 0 will disable the glitch filter. Glitches equal to or less than FILTSDA cycles long will be filtered out and ignored. The latency through the glitch filter is equal to FILTSDA cycles and must be configured less than the minimum SCL low or high period.	

	The glitch filter cycle count is not affected by the PRESCALE configuration and is automatically bypassed in High Speed mode.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	Origin	Flagchip

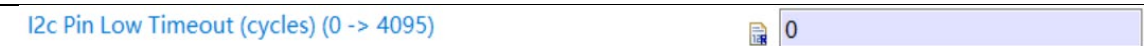
3.2.5.8 I2cGlitchFilterSCL

Container	I2cGlitchFilterSCL	
Description	Glitch Filter SCL: Configures FCI2C_MCFGR2[FILTSCS] Configures the I2c master digital glitch filters for SCL input, a configuration of 0 will disable the glitch filter. Glitches equal to or less than FILTSCS cycles long will be filtered out and ignored. The latency through the glitch filter is equal to FILTSCS cycles and must be configured less than the minimum SCL low or high period. The glitch filter cycle count is not affected by the PRESCALE configuration and is automatically bypassed in High Speed mode.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	Origin	Flagchip

3.2.5.9 I2cBusIdleTimeout

Container	I2cBusIdleTimeout	
Description	Bus Idle Timeout: Configures FCI2C_MCFGR2[BUSIDLE] Configures the bus idle timeout period in clock cycles. If both SCL and SDA are high for longer than BUSIDLE cycles, then the I2c bus is assumed to be idle and the master can generate a START condition. When set to zero, this feature is disabled.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	Origin	Flagchip

3.2.5.10 I2cPinLowTimeout

Container	I2cPinLowTimeout	
Description	Pin Low Timeout: Configures FCI2C_MCFGR3[PINLOW] Configures the pin low timeout flag in clock cycles. If SCL and/or SDA is low for longer than (PINLOW * 256) cycles then PLTF is set. When set to zero, this feature is disabled.	
Screenshot		
Properties	Property	Value
	Type	INTEGER

	Origin	Flagchip
--	--------	----------

3.2.5.11 I2cDataValidDelay

Container	I2cDataValidDelay	
Description	Data Valid Delay: Configures FCI2C_MCCR0[DATAVD] Minimum number of cycles (minus one) that is used as the data hold time for SDA. Must be configured less than the minimum SCL low period.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	Origin	Flagchip

3.2.5.12 I2cSetupHoldDelay

Container	I2cSetupHoldDelay	
Description	Setup Hold Delay: Configures FCI2C_MCCR0[SETHOLD] Minimum number of cycles (minus one) that is used by the master as the setup and hold time for a (repeated) START condition and setup time for a STOP condition. The setup time is extended by the time it takes to detect a rising edge on the external SCL pin. Ignoring any additional board delay due to external loading, this is equal to $(2 + \text{FILTSCl}) / 2^{\text{PRESCALE}}$ cycles.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	Origin	Flagchip

3.2.5.13 I2cClockHighPeriod

Container	I2cClockHighPeriod	
Description	Clock High Period: Configures FCI2C_MCCR0[CLKHI] Minimum number of cycles (minus one) that the SCL clock is driven high by the master. The SCL high time is extended by the time it takes to detect a rising edge on the external SCL pin. Ignoring any additional board delay due to external loading, this is equal to $(2 + \text{FILTSCl}) / 2^{\text{PRESCALE}}$ cycles	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	Origin	Flagchip

3.2.5.14 I2cClockLowPeriod

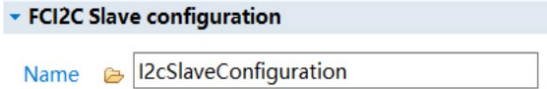
Container	I2cClockLowPeriod	
Description	Clock Low Period: Configures FCI2C_MCCR0[CLKLO] Minimum number of cycles (minus one) that the SCL clock is driven low by the master.	

	This value is also used for the minimum bus free time between a STOP and a START condition	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	Origin	Flagchip

3.2.5.15 I2cBaudRate

Container	I2cClockRef	
Description	Calculated as $\text{Frequency}/(\text{CLKLO}+\text{CLKHI}+2+\text{FILTSCl})/(2^{\text{PRESCALER}})$ The functional clock must be at least 8 times faster than the I2c bus bandwidth.	
Screenshot		
Properties	Property	Value
	Type	FLOAT
	Origin	Flagchip

3.2.6 I2cSlaveConfiguration

Container	I2cSlaveConfiguration	
Description	This container contains the configuration (parameters) of the Slave Channel.	
Screenshot		
Properties	Property	Value
	Type	Container

3.2.6.1 I2cSlaveAddress

Container	I2cSlaveAddress	
Description	The address of the slave: Configures FCI2C_SAMR[ADDR0] Configures the I2c slave address.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	Origin	Flagchip

3.2.6.2 I2cSlaveFilterEnable

Container	I2cSlaveFilterEnable	
Description	I2cSlaveFilterEnable configures FCI2C_SCR[FILTEN] Check to enable the digital filter and output delay counter for slave mode.	
Screenshot		
Properties	Property	Value

	Type	BOOLEAN
	Origin	Flagchip
	Default	TRUE

3.2.6.3 I2cSlaveAckStall

Container	I2cSlaveAckStall	
Description	I2cSlaveAckStall configures FCI2C_SCFGR1[ACKSTALL] Enables SCL clock stretching during slave-transmit address byte(s) and slave-receiver address and data byte(s) to allow software to write the Transmit ACK Register before the ACK or NACK is transmitted. Clock stretching occurs when transmitting the 9th bit and is therefore not compatible with high speed mode. When ACKSTALL is enabled, there is no need to set either RXSTALL or ADRSTALL.	
Screenshot	I2c Enable ACK SCL Stall 	
Properties	Property	Value
	Type	BOOLEAN
	Origin	Flagchip
	Default	FALSE

3.2.6.4 I2cSlaveTxStall

Container	I2cSlaveTxStall	
Description	I2cSlaveTxStall configures FCI2C_SCFGR1[TXSTALL] Enables SCL clock stretching when the transmit data flag is set during a slave-transmit transfer. Clock stretching occurs following the 9th bit and is therefore compatible with high speed mode.	
Screenshot	I2c Enable TX Data SCL Stall 	
Properties	Property	Value
	Type	BOOLEAN
	Origin	Flagchip
	Default	FALSE

3.2.6.5 I2cSlaveRxStall

Container	I2cSlaveRxStall	
Description	I2cSlaveRxStall configures FCI2C_SCFGR1[RXSTALL] Enables SCL clock stretching when the receive data flag is set during a slave-receive transfer. Clock stretching occurs following the 9th bit and is therefore compatible with high speed mode.	
Screenshot	I2c Enable RX Data SCL Stall 	
Properties	Property	Value
	Type	BOOLEAN
	Origin	Flagchip
	Default	FALSE

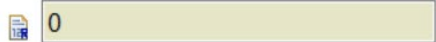
3.2.6.6 I2cSlaveAdrStall

Container	I2cSlaveAdrStall	
Description	I2cSlaveAdrStall configures FCI2C_SCFGR1[ADRSTALL] Enables SCL clock stretching when the address valid flag is asserted. Clock stretching only occurs following the 9th bit and is therefore compatible with high speed mode.	
Screenshot	I2c Enable Address SCL Stall 	
Properties	Property	Value
	Type	BOOLEAN
	Origin	Flagchip
	Default	FALSE

3.2.6.7 I2cGlitchFilterSDA

Container	I2cGlitchFilterSDA	
Description	Glitch Filter SDA: Configures FCI2C_MCFGR2[FILTSDA] Configures the I2c master digital glitch filters for SDA input, a configuration of 0 will disable the glitch filter. Glitches equal to or less than FILTSDA cycles long will be filtered out and ignored. The latency through the glitch filter is equal to FILTSDA cycles and must be configured less than the minimum SCL low or high period.	
Screenshot	I2c Glitch Filter SDA (cycles) (0 -> 15) 	
Properties	Property	Value
	Type	INTEGER
	Origin	Flagchip

3.2.6.8 I2cGlitchFilterSCL

Container	I2cGlitchFilterSCL	
Description	Glitch Filter SCL: Configures FCI2C_MCFGR2[FILTSCl] Configures the I2c master digital glitch filters for SCL input, a configuration of 0 will disable the glitch filter. Glitches equal to or less than FILTSCl cycles long will be filtered out and ignored. The latency through the glitch filter is equal to FILTSCl cycles and must be configured less than the minimum SCL low or high period. The glitch filter cycle count is not affected by the PRESCALE configuration and is automatically bypassed in High Speed mode.	
Screenshot	I2c Glitch Filter SCL (cycles) (0 -> 15) 	
Properties	Property	Value
	Type	INTEGER
	Origin	Flagchip

3.2.6.9 I2cDataValidDelay

Container	I2cDataValidDelay
------------------	-------------------

Description	Data Valid Delay: Configures FCI2C_SCFGR2[DATAVD] Configures the SDA data valid delay time for the I2c slave equal to FILTSCL+DATAVD+3 cycles. This data valid delay must be configured to less than the minimum SCL low period. The I2c slave data valid delay time is not affected by the PRESCALE configuration, and is disabled in high speed mode.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	Origin	Flagchip

3.2.6.10 I2cClockHoldPeriod

Container	I2cClockHoldPeriod	
Description	Clock Hold Period: Configures FCI2C_SCFGR2[CLKHOLD] Minimum number of cycles (minus one) that the SCL clock is driven high by the master. Configures the minimum clock hold time for the I2c slave, when clock stretching is enabled. The minimum hold time is equal to CLKHOLD+3 cycles. The I2c slave clock hold time is not affected by the PRESCALE configuration, and is disabled in high speed mode.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	Origin	Flagchip

3.2.7 CommonPublishedInformation

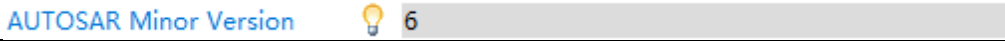
Container	CommonPublishedInformation	
Description	Common container, aggregated by all modules. It contains published information about vendor and versions.	
Screenshot		
Properties	Property	Value
	Type	Container

3.2.7.1 ArReleaseMajorVersion

Variable	ArReleaseMajorVersion	
Description	Major version number of AUTOSAR specification on which the appropriate implementation is based.	
Screenshot		
Properties	Property	Value
	Type	INTEGER

	Origin	Flagchip
--	--------	----------

3.2.7.2 ArReleaseMinorVersion

Variable	ArReleaseMinorVersion	
Description	Minor version number of AUTOSAR specification on which the appropriate implementation is based.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	Origin	Flagchip

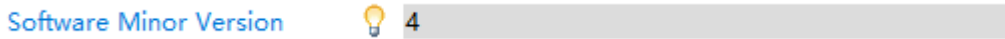
3.2.7.3 ArReleaseRevisionVersion

Variable	ArReleaseRevisionVersion	
Description	Revision version number of AUTOSAR specification on which the appropriate implementation is based.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	Origin	Flagchip

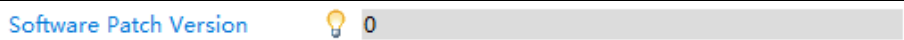
3.2.7.4 SwMajorVersion

Variable	SwMajorVersion	
Description	Major version number of the vendor specific implementation of the module. The numbering is vendor specific.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	Origin	Flagchip

3.2.7.5 SwMinorVersion

Variable	SwMinorVersion	
Description	Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	Origin	Flagchip

3.2.7.6 SwPatchVersion

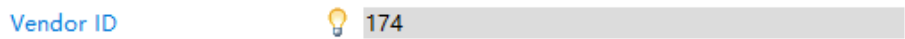
Variable	SwPatchVersion	
Description	Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.	
Screenshot		

Properties	Property	Value
	Type	INTEGER
	Origin	Flagchip

3.2.7.7 ModuleId

Variable	ModuleId	
Description	Module ID of this module from Module List.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	Origin	Flagchip

3.2.7.8 VendorId

Variable	VendorId	
Description	Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	Origin	Flagchip

Chapter 4 Configuration Guides

4.1 Configuration Item Constraint

- 1) If the I2C notification is needed, you should not set notification pointer to NULL_PTR.

I2c

Name  I2c

General I2cEcucPartitionRef I2cChannel Published Information

Config Variant  VariantPostBuild 

GeneralConfiguration

Name  GeneralConfiguration

I2c Development Error Detection	 ☐ 	I2c Multicore Support	 ☐ 
I2c Timeout Duration (1 -> 65535)	 1000 		
Provide I2c VersionInfo Api	 ☒ 	I2c DMA Used	 ☒ 
 I2cErrorNotification	 I2c_ErrorNotification 		
 I2cMasterTransmitCompleteNotification	 I2c_MasterTransmitCompleteNotification 		
 I2cMasterReceiveCompleteNotification	 I2c_MasterReceiveCompleteNotification 		
 I2cSlaveAddressMatchNotification	 I2c_SlaveAddressMatchNotification 		
 I2cSlaveTransmitCompleteNotification	 I2c_SlaveTransmitCompleteNotification 		
 I2cSlaveReceiveCompleteNotification	 I2c_SlaveReceiveCompleteNotification 		
 I2cSlaveByteReceiveNotification	 I2c_SlaveByteReceiveNotification 		

I2cGlobalConfig

Name  I2cGlobalConfig

- 2) If DMA mode for i2c master async transmit is needed, you should enable the item "I2c DMA Used".

I2c

Name  I2c

General I2cEcucPartitionRef I2cChannel Published Information

Config Variant  VariantPostBuild 

GeneralConfiguration

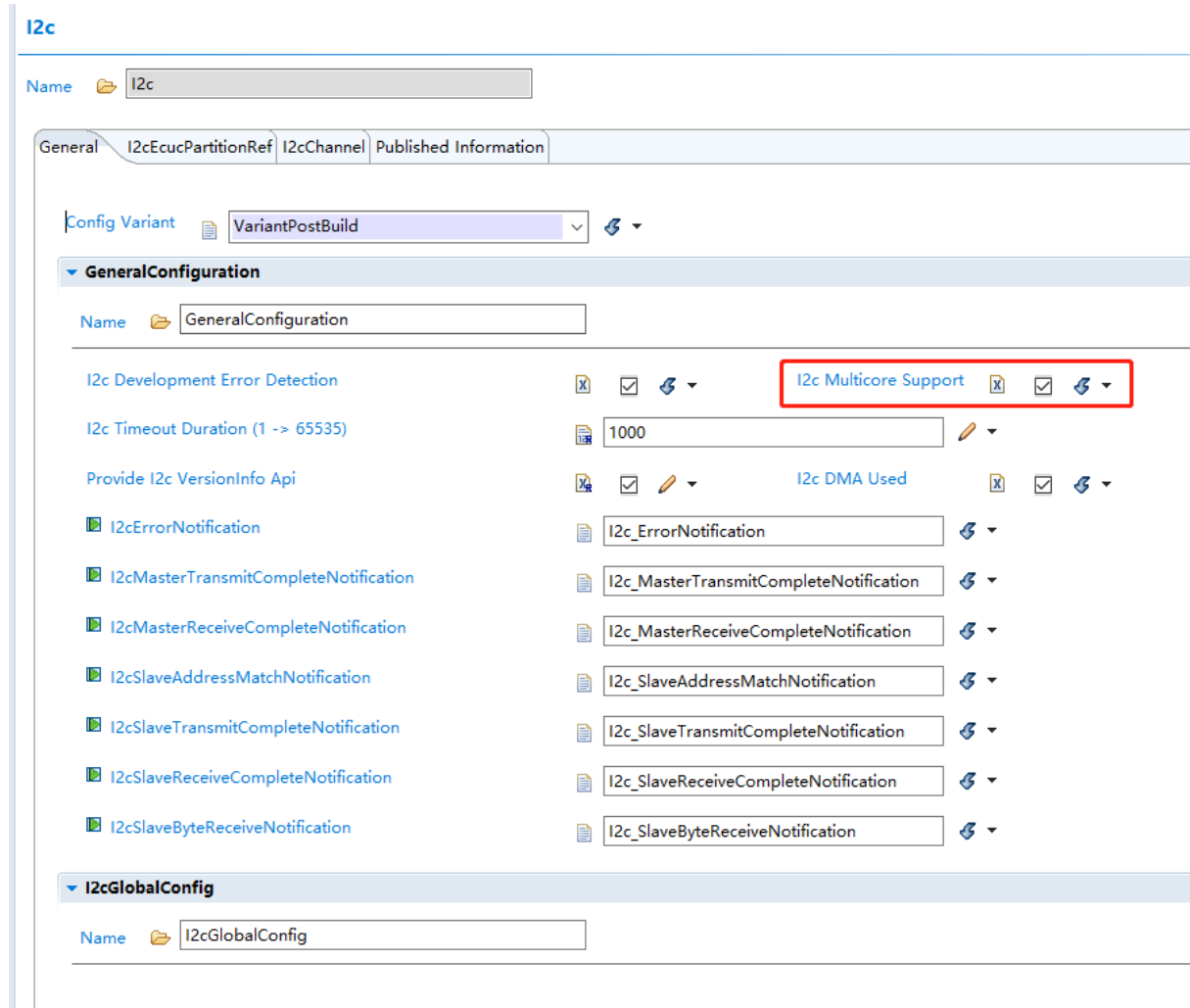
Name  GeneralConfiguration

I2c Development Error Detection	 ☐ 	I2c Multicore Support	 ☐ 
I2c Timeout Duration (1 -> 65535)	 1000 		
Provide I2c VersionInfo Api	 ☒ 	I2c DMA Used	 ☒ 
 I2cErrorNotification	 I2c_ErrorNotification 		
 I2cMasterTransmitCompleteNotification	 I2c_MasterTransmitCompleteNotification 		
 I2cMasterReceiveCompleteNotification	 I2c_MasterReceiveCompleteNotification 		
 I2cSlaveAddressMatchNotification	 I2c_SlaveAddressMatchNotification 		
 I2cSlaveTransmitCompleteNotification	 I2c_SlaveTransmitCompleteNotification 		
 I2cSlaveReceiveCompleteNotification	 I2c_SlaveReceiveCompleteNotification 		
 I2cSlaveByteReceiveNotification	 I2c_SlaveByteReceiveNotification 		

I2cGlobalConfig

Name  I2cGlobalConfig

- 3) If i2c is needed to use for multicore, you should enable the item "I2c Multicore Support".



4.2 I2C Usage Common Steps

Basically, the I2C module can be configured by following the below steps:

- 1) Configure I2C channels in tab "I2cChannel".

I2cChannel*						
Index	Name	I2c Channel ID	I2c hardware channel	I2c master/slave configuration	I2c pin configuration	
0	I2cChannel_0	0	FCI2C_0	MASTER_MODE	PINCFG_2PIN_OPEN_DRAIN	
1	I2cChannel_1	1	FCI2C_1	MASTER_MODE	PINCFG_2PIN_OPEN_DRAIN	

- 2) Double-click the item, and select i2c mode.

I2cChannel

Name  I2cChannel_0

General

I2c Channel ID  0 

I2c hardware channel  FCI2C_1 

I2c master/slave configuration  MASTER_MODE 

I2c pin configuration  PINCFG_2PIN_OPEN_DRAIN 

 I2cCtrlEcucPartitionRef 

- 3) If master mode and item "I2c DMA Used" are enabled, you can select DMA mode for item "I2c Async Mode" and select DMA channel which is set in the DMA module.

FCI2C Master configuration

Name  I2cMasterConfiguration

I2c Master enabled in debug mode  

 I2cClockRef*  /Mcu/Mcu/McuModuleConfiguration/MCU_Demo_FOSC24M/McuClockReferencePoint_FCIIC0

I2c Async Mode  DMA 

 I2c TX DMA Channel  /DMA/DMA/DMAConfigSet/DmaChannelConfig_3

 I2c RX DMA Channel  /DMA/DMA/DMAConfigSet/DmaChannelConfig_4

- 4) If slave mode, you must set the slave address for I2c.

FCI2C Slave configuration

Name  I2cSlaveConfiguration

I2c Slave Address (0 -> 1023)  35 

I2c Enable Slave Filter   I2c Enable ACK SCL Stall  

I2c Enable TX Data SCL Stall   I2c Enable RX Data SCL Stall  

I2c Enable Address SCL Stall  

I2c Glitch Filter SDA (cycles) (0 -> 15)  0 

I2c Glitch Filter SCL (cycles) (0 -> 15)  0 

I2c Data Valid Delay (cycles)  0 

I2c Clock Hold Period (cycles) (0 -> 15)  0 

- 5) Generate configuration files.